A Project Report

On

Freshers Guide

Submitted by

Aditi Bodkhe

Payal Bhagat

Jaya Bhute

For the Partial Fulfillment of the Degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted to

SANT GADGE BABA AMRAVATI UNIVERSITY, AMRAVATI

Under the Supervision of

Prof. Prabhakar Wadatkar

DEPARTMENT OF BACHELOR OF COMPUTER APPLICATIONS

SARASWATI COLLEGE, SHEGAON

2025-26

SARASWATI COLLEGE, SHEGAON

DEPARTMENT OF BACHELOR OF COMPUTER

CERTIFICATE

This is to certify that, Miss. Aditi Vishwanath Bodkhe, Miss. Payal Suresh Bhagat and Miss. Jaya Bhagwant Bhute have successfully completed Their project work entitled "Freshers Guide Project" for the partial fullfillment of degree of Bachelor of Computer Application to Sant Gadge Baba Amravati University, Amravati during the academic year 2023 – 2026.

Supervisor

Internal Examiner External Examiner

Head of the Department Principal

DECLARATION BY THE STUDENT

We Aditi Vishwanath Bodkhe, Payal Suresh Bhagat and Jaya Bhagwant Bhute hereby declare that we have completed our project work for the partial fullfillment of the degree of Bachelor of Computer Applications of Sant Gadge Baba Amravati University, Amravati during the academic year 2023-2025. This project work fulfills all the requirements laid down by Sant Gadge Baba Amravati University, Amravati. If the above mentioned project proves fake or does not fullfills prerequisites of Sant Gadge Baba Amravati University, we will be solely responsible for all the consequences.

Date: / /2026 Signature of Student

Aditi Vishwanath Bodkhe

Payal Suresh Bhagat

Jaya Bhagwant Bhute

ACKNOWLEDGMENT

Inspiration and guidance are valuable in every aspect of life, especially in the field of academics. I would like to express my sincere gratitude to my respected project guide, Prof. Prabhakar Wadatkar for their invaluable guidance and continuous support from the commencement to the completion of this project. I am also grateful to all the teachers of our department who encouraged me throughout this journey and provided their insights to enhance my learning experience. Furthermore, I remain indebted to my parents and friends for their constant support, motivation, and encouragement, which played a crucial role in the successful completion of this project.

Date: / /2026

Signature of Student

Aditi Vishwanath Bodkhe

Payal Suresh Bhagat

Jaya Bhagwant Bhute

Table of Contents

Title Page		
Certificate		
Declaration		
Acknowledgement		
Table of Contents		
Abstract		
Chapters		
Sr. No.	**Contents**	**Page No.**
1	Introduction	1
1.1	About the Project	1
1.2	Objectives	2
1.3	Scope of the Project	2
2	Project Overview	3
3	Home Page	4
4	Register Page	6
5	Login Page	10
6	Dashboard Page	13
6.1	Resume Building	16
6.2	Interview Preparation	19
6.3	Communication	22
6.4	Work Culture	25
6.5	Office Etiquette	28
6.6	Salary Module	30
7	DataBase Implementation	34
7.1	MySQL Database	35

7.2	Backend server	36
7.3	Database Connection	38
8	API IMPLEMENTATION	39
8.1	GET API	40
8.2	POST API	41
8.3	PUT API	43
8.4	DELETE API	44
9	Advantages of the system	45
10	Future Scope	46
11	Conclusion	47
12	References	48

Abstract

The **"Freshers Guide"** is a web-based application developed to provide career guidance and professional awareness to students and fresh graduates. In today's competitive job market, academic knowledge alone is not sufficient for securing and sustaining a job. Many freshers lack clarity about resume preparation, interview handling, communication skills, salary structure, workplace behavior, and corporate culture. This platform is designed to bridge that gap and support students in preparing for professional life.

The website begins with the **Resume Building module**, where a structured resume format is provided to guide students. Users can enter their personal, educational, and professional details, preview the resume, and download it to their computer. This helps freshers understand the correct format and structure of a professional resume.

The next section is **Interview Preparation**, which includes 20 Common Questions, 20 HR Questions, and 20 Technical Questions, each supported with model answers. It also provides interview tips, a preparation checklist, and a section on common mistakes to avoid. This module helps students prepare confidently and systematically for interviews.

The **Communication module** focuses on improving professional interaction skills. It covers verbal communication, non-verbal communication, active listening skills, and feedback skills. This guidance helps freshers communicate effectively during interviews and in workplace environments.

The **Work Culture module** provides awareness about teamwork, discipline, corporate values, work-life balance, learning and development, diversity and inclusion, and employee wellbeing and support. It prepares students to adapt to the organizational environment and maintain a healthy professional life.

The **Office Etiquette module** explains professional etiquette, social etiquette, digital etiquette (such as email and online communication), and understanding workplace hierarchy. This module helps freshers behave appropriately and respectfully in corporate settings.

Finally, the **Salary Module** provides guidance on company-wise salary information, salary components table, salary slip explanation, salary calculator, and salary negotiation guide. This helps students understand CTC structure, in-hand salary, and compensation discussions clearly.

Developed using React JS, HTML, CSS, and JavaScript, the Freshers Guide offers a responsive and user-friendly interface. It does not provide job placement services but acts as a comprehensive career guidance platform.

In conclusion, the Freshers Guide prepares students not only to secure a job but also to understand workplace expectations and survive confidently in the corporate world.

1. Introduction

1.1 About the Project

The Freshers Guide is a web-based application developed using modern web technologies such as React JS, HTML, CSS, and JavaScript. The platform is designed to provide career guidance to students and fresh graduates in a structured and user-friendly manner.

The website consists of multiple modules including:

- Resume Building
- Interview Preparation
- Communication Skills
- Work Culture
- Office Etiquette
- Salary Module

Each module focuses on a specific aspect of career preparation and professional awareness. The Resume Builder provides a structured format where users can enter their details and download their resume. The Interview Preparation module includes 20 Common Questions, 20 HR Questions, and 20 Technical Questions with model answers, along with interview tips, checklist, and common mistakes. Other modules provide guidance about workplace communication, corporate culture, etiquette, and salary understanding.

The project does not provide job placement services but serves as an informational and guidance platform for career readiness.

1.2 Objectives of the Project

The main objectives of this project are:

- To guide freshers in understanding the professional office environment
- To explain basic office etiquette and work culture
- To help freshers understand salary structure and CTC
- To improve professional communication skills
- To provide an easy-to-use resume builder
- To help students practice common interview questions
- To design a clean, responsive, and beginner-friendly website
- To develop a project suitable for BCA final year evaluation

1.3 Scope of the Project

Provides structured career guidance for students and fresh graduates.

- Helps in resume building, interview preparation, and communication skill development.
- Includes professional awareness modules such as work culture and office etiquette.
- Builds understanding of workplace discipline, hierarchy, and professional behavior.
- Explains salary concepts like CTC, salary components, salary slips, and negotiation ti

2. Project Overview

The “Freshers Guide” is a career guidance **website** developed to help students and fresh graduates prepare for jobs, placements, and interviews. In today’s competitive job market, many students face confusion regarding resume preparation, salary structure, interview etiquette, communication skills, and company work culture. Due to lack of awareness, freshers often struggle not only to secure a job but also to adjust to the corporate environment. This website is designed to provide step-by-step guidance and reduce that confusion.

The website consists of several important sections. The Dashboard serves as the main navigation page from where users can access different modules such as the Salary Module, Resume Builder, Interview Preparation, and Work Culture guidance.

The Salary Module includes an interactive Salary Calculator where users can enter their CTC (Cost to Company) and calculate their estimated monthly in-hand salary. This helps freshers understand salary breakdown and financial concepts clearly.

The Resume Builder section provides a structured resume format. Users can enter their details such as education, skills, projects, and certifications, preview the resume, and download it to their computer. This helps them create a professional resume easily.

The Interview Preparation section provides guidance on common HR questions, communication tips, and professional behavior during interviews. It supports students in preparing confidently for campus placements and company interviews.

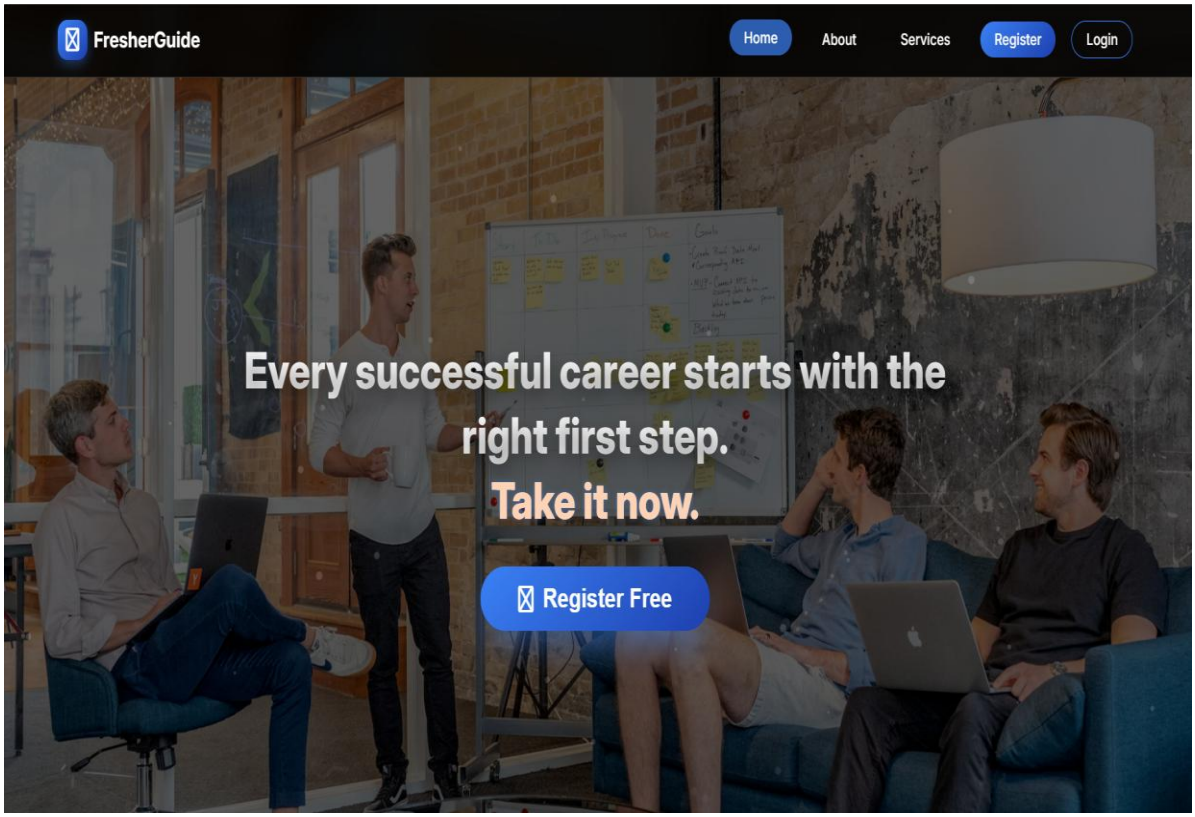
The Work Culture and Office Etiquette section provides information about professional behavior, workplace communication, and corporate expectations. This prepares freshers to adjust smoothly in a professional environment.

Overall, the Freshers Guide is an informative and guidance-based website that helps students prepare for employment and understand workplace expectations. It serves as a structured career preparation platform aimed at building confidence and improving readiness before entering the corporate world.

3. Home Page

The home page presents a professional and modern banner introducing the FresherGuide platform. It features a dark-themed navigation bar at the top with links to Home, About, Services, along with Register and Login buttons for user access.

The center of the page displays a motivational headline: “Every successful career starts with the right first step. Take it now.” This creates an inspiring first impression for freshers and job seekers.



HTML CODE

index.html ✕ # style.css

assets > Homepage > index.html > html > body > div#authModal.auth-modal > div.auth-card-modern > div.auth-body > div#loginPanel.auth-form-panel > form > div.input-group-modern > input#loginEm

```html
<html lang="en">
<body>

<!-- AUTH MODAL - BLUE THEME -->
<div id="authModal" class="auth-modal">
    <div class="close-modal" onclick="closeAuthModal()"><i class="fas fa-times"></i></div>
    <div class="auth-card-modern">
        <div class="auth-header-modern">
            <div class="logo-badge"><i class="fas fa-graduation-cap"></i></div>
            <h3>Welcome to FresherGuide</h3>
            <p>Start your journey to career success</p>
        </div>
        <div class="auth-tabs">
            <div class="auth-tab active" onclick="switchAuthTab('register')">📝 Register</div>
            <div class="auth-tab" onclick="switchAuthTab('login')">🔐 Login</div>
        </div>
        <div class="auth-body">
            <div id="registerPanel" class="auth-form-panel active">
                <form onsubmit="return handleRegister(event)">
                    <div class="input-group-modern">
                        <i class="fas fa-user"></i>
                        <input type="text" id="regName" placeholder="Full Name">
                        <div class="error-text" id="regNameError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-envelope"></i>
                        <input type="email" id="regEmail" placeholder="Email Address">
                        <div class="error-text" id="regEmailError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-lock"></i>
                        <input type="password" id="regPassword" placeholder="Password (min 6 characters)">
                        <span class="password-toggle-icon" onclick="togglePassword('regPassword', this)"><i class="far fa-eye-slash"></i></span>
                        <div class="error-text" id="regPasswordError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-check-circle"></i>
```

Update is ready, click to restart. Ln 263, Col 89 Spaces: 4 UTF-8 LF {} HTML

CSS CODE

index.html # style.css ✕

assets > Homepage > style.css > .dynamic-bg-slideshow

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Inter', sans-serif;
    color: #1e2e44;
    scroll-behavior: smooth;
    overflow-x: hidden;
}

/* ========= DYNAMIC BACKGROUND SLIDESHOW ========= */
.dynamic-bg-slideshow {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: -2;
    overflow: hidden;
}

.bg-slide {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
    opacity: 0;
    transition: opacity 1.8s ease-in-out;
    animation: kenBurns 20s ease-in-out infinite alternate;
```

JAVASCRIPT CODE

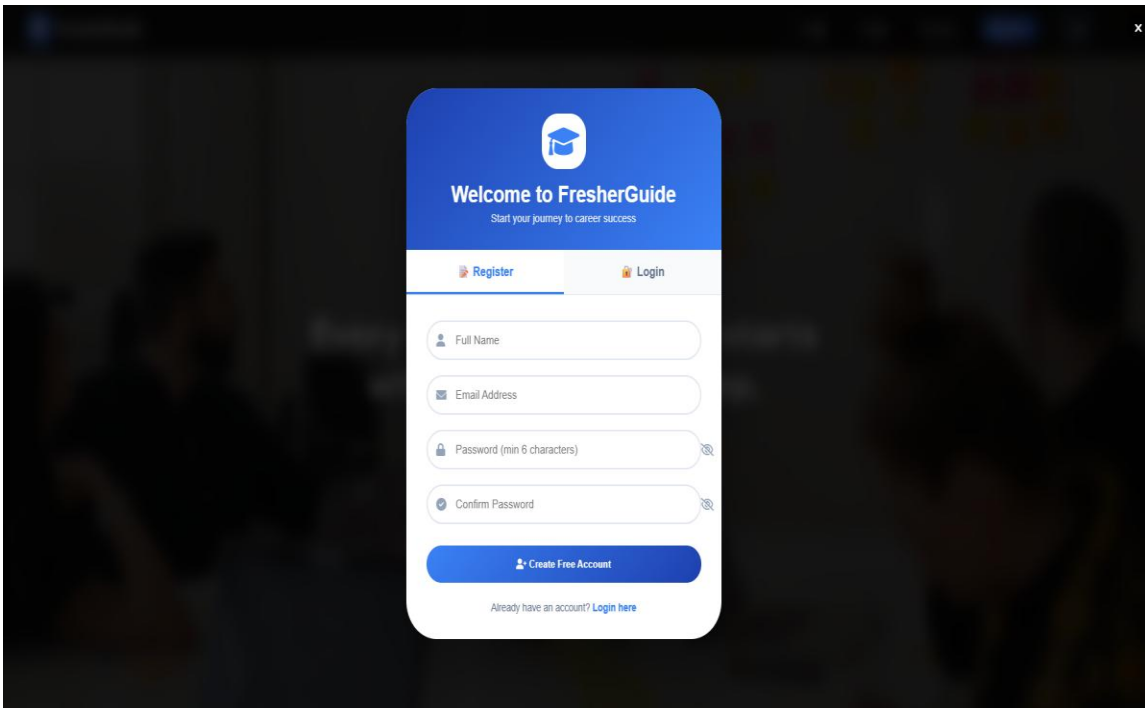
index.html script.js

ssets > Homepage > script.js > ...

```javascript
// User Database (persistent in localstorage)
let users = JSON.parse(localStorage.getItem('fresherguide_users')) || [];

function saveUsers() {
    localStorage.setItem('fresherguide_users', JSON.stringify(users));
}

// Background slideshow
const slides = document.querySelectorAll('.bg-slide');
let currentSlide = 0;
if(slides.length > 0) {
    slides[0].classList.add('active');
    setInterval(() => {
        slides[currentSlide].classList.remove('active');
        currentSlide = (currentSlide + 1) % slides.length;
        slides[currentSlide].classList.add('active');
    }, 5000);
}

// Create floating particles
function createParticles() {
    const container = document.getElementById('particles');
    if(!container) return;
    for(let i = 0; i < 50; i++) {
        const p = document.createElement('div');
        p.classList.add('particle');
        p.style.width = Math.random() * 6 + 2 + 'px';
        p.style.height = p.style.width;
        p.style.left = Math.random() * 100 + '%';
        p.style.animationDuration = Math.random() * 15 + 8 + 's';
        p.style.animationDelay = Math.random() * 10 + 's';
        container.appendChild(p);
    }
}
createParticles();

// Toast Notification
```

4. Registration page

This page displays a clean and modern registration form for Fresher Guide. It includes input fields for Full Name, Email Address, Password, and Confirm Password. The design features a blue header with a welcome message and tabs for Register and Login. A prominent “Create Free Account” button is provided to complete the sign-up process



HTML CODE

```html
<html lang="en">
<body>

<!-- AUTH MODAL - BLUE THEME -->
<div id="authModal" class="auth-modal">
    <div class="close-modal" onclick="closeAuthModal()"><i class="fas fa-times"></i></div>
    <div class="auth-card-modern">
        <div class="auth-header-modern">
            <div class="logo-badge"><i class="fas fa-graduation-cap"></i></div>
            <h3>Welcome to FresherGuide</h3>
            <p>Start your journey to career success</p>
        </div>
        <div class="auth-tabs">
            <div class="auth-tab active" onclick="switchAuthTab('register')">📝 Register</div>
            <div class="auth-tab" onclick="switchAuthTab('login')">🔐 Login</div>
        </div>
        <div class="auth-body">
            <div id="registerPanel" class="auth-form-panel active">
                <form onsubmit="return handleRegister(event)">
                    <div class="input-group-modern">
                        <i class="fas fa-user"></i>
                        <input type="text" id="regName" placeholder="Full Name">
                        <div class="error-text" id="regNameError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-envelope"></i>
                        <input type="email" id="regEmail" placeholder="Email Address">
                        <div class="error-text" id="regEmailError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-lock"></i>
                        <input type="password" id="regPassword" placeholder="Password (min 6 characters)">
                        <span class="password-toggle-icon" onclick="togglePassword('regPassword', this)"><i class="far fa-eye-slash"></i></span>
                        <div class="error-text" id="regPasswordError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-check-circle"></i>
```

CSS CODE

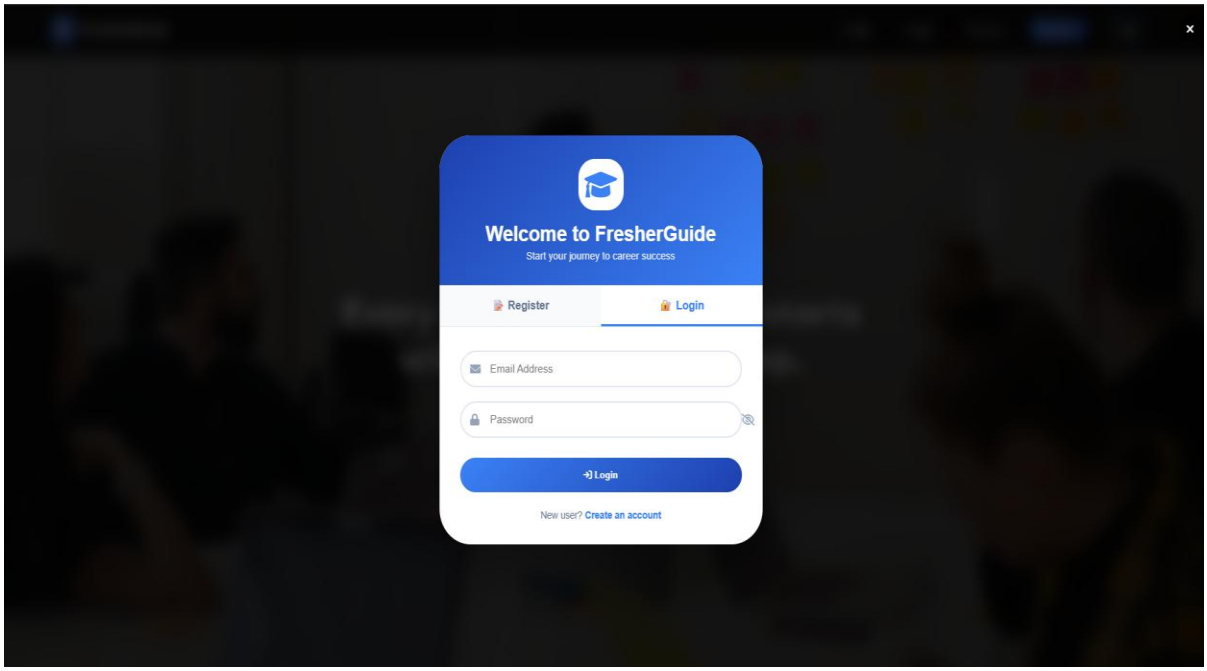
```css
/* Auth Modal - BLUE THEME */
.auth-modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.75);
    backdrop-filter: blur(10px);
    z-index: 2000;
    align-items: center;
    justify-content: center;
}

.auth-modal.active {
    display: flex;
    animation: fadeIn 0.3s ease;
}

@keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
}

.auth-card-modern {
    max-width: 500px;
    width: 90%;
    background: white;
    border-radius: 48px;
    overflow: hidden;
    box-shadow: 0 40px 70px rgba(0, 0, 0, 0.3);
    animation: slideUp 0.4s ease;
}

.auth-header-modern {
```

JAVASCRIPT CODE

```javascript
// Register Handler
// Replace your existing handleRegister function
async function handleRegister(e) {
    e.preventDefault();
    clearAuthErrors();

    const name = document.getElementById('regName').value.trim();
    const email = document.getElementById('regEmail').value.trim();
    const password = document.getElementById('regPassword').value;

    try {
        const res = await fetch('http://localhost:8080/api/auth/register', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ name, email, password })
        });

        const data = await res.json();

        if (res.ok) {
            showToast(`✅ Welcome ${name}! Registration successful`, 'success');
            setTimeout(() => {
                window.location.href = "../assets/Dashboard/index.html";
            }, 1200);
        } else {
            showToast(data.error || 'Registration failed', 'warning');
        }
    } catch (err) {
        showToast('Cannot connect to server. Is backend running?', 'warning');
    }
}

// Replace your existing handleLogin function
async function handleLogin(e) {
    e.preventDefault();
```

5. Login Page

The FresherGuide login page features a clean, split-screen design aimed at helping new professionals start their career journeys. It provides dedicated fields for an Email Address and Password, centered around a prominent blue Login button. For new users, a clear Register link is included to facilitate easy account creation



HTML CODE

```html
<html lang="en">
<body>
<div id="authModal" class="auth-modal">
    <div class="auth-card-modern">
        <div class="auth-body">

            <div id="loginPanel" class="auth-form-panel">
                <form onsubmit="return handleLogin(event)">
                    <div class="input-group-modern">
                        <i class="fas fa-envelope"></i>
                        <input type="email" id="loginEmail" placeholder="Email Address">
                        <div class="error-text" id="loginEmailError"></div>
                    </div>
                    <div class="input-group-modern">
                        <i class="fas fa-lock"></i>
                        <input type="password" id="loginPassword" placeholder="Password">
                        <span class="password-toggle-icon" onclick="togglePassword('loginPassword', this)"><i class="far fa-eye-slash"></i></span>
                        <div class="error-text" id="loginPasswordError"></div>
                    </div>
                    <button type="submit" class="auth-submit-btn"><i class="fas fa-arrow-right-to-bracket"></i> Login</button>
                </form>
                <div class="auth-footer">New user? <span onclick="switchAuthTab('register')">Create an account</span></div>
            </div>
        </div>
    </div>
</div>

<!-- External JavaScript -->
<script src="script.js"></script>
</body>
</html>
```

CSS CODE

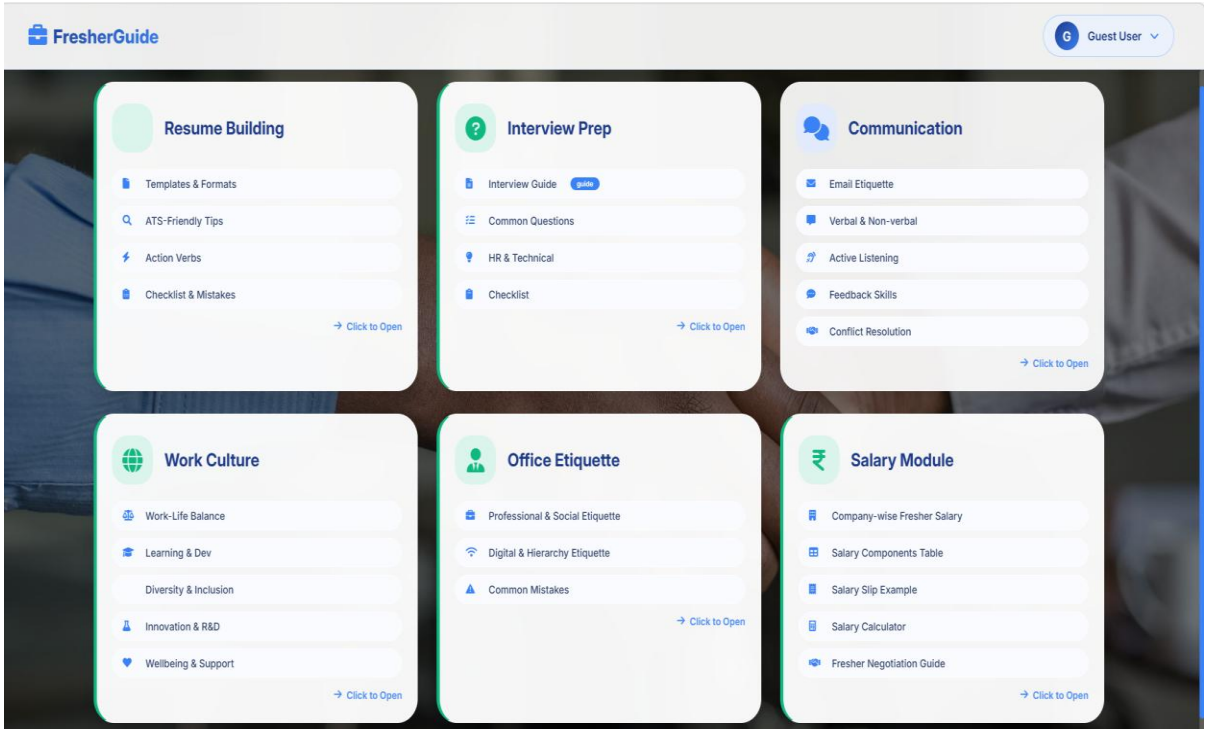
```css
.error-text {
    color: #ef4444;
    font-size: 0.7rem;
    margin-top: 0.3rem;
    margin-left: 1rem;
    display: none;
}

.error-text.show {
    display: block;
}

.auth-submit-btn {
    width: 100%;
    padding: 1rem;
    background: linear-gradient(135deg, #3b82f6, #1e40af);
    color: white;
    border: none;
    border-radius: 50px;
    font-weight: 700;
    cursor: pointer;
    margin-top: 0.5rem;
    transition: 0.2s;
}

.auth-submit-btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 10px 20px rgba(59,130,246,0.3);
}

.auth-footer {
    text-align: center;
    margin-top: 1.5rem;
    font-size: 0.85rem;
    color: #64748b;
}
```

JAVASCRIPT CODE

```javascript
// Replace your existing handleLogin function
async function handleLogin(e) {
    e.preventDefault();
    clearAuthErrors();

    const email = document.getElementById('loginEmail').value.trim();
    const password = document.getElementById('loginPassword').value;

    try {
        const res = await fetch('http://localhost:8080/api/auth/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, password })
        });

        const data = await res.json();

        if (res.ok) {
            showToast(`🎉 Welcome back, ${data.user.name}!`, 'success');
            setTimeout(() => {
                window.location.href = "../assets/Dashboard/index.html";
            }, 1000);
        } else {
            showToast(data.error || 'Invalid credentials', 'warning');
        }
    } catch (err) {
        showToast('Cannot connect to server', 'warning');
    }
}
```

6. Dashboard Page

The FresherGuide dashboard is an all-in-one career toolkit designed to help graduates transition into their first jobs. It provides modules for resume building, interview prep, and communication skills to boost your professional profile. Additionally, it offers practical advice on office etiquette, work culture, and salary negotiation to ensure a smooth start to your career.



HTML CODE

```html
index.html ...\Homepage    index.html ...\Dashboard
ssets > Dashboard > index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
    <title>FresherGuide · Career Dashboard | Full Screen</title>
    <!-- Font Awesome 6 -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <!-- Google Fonts -->
    <link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,300;14..32,400;14..32,500;14..32,600;14..32,700;14..32,800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="dashboard">
    <!-- Navigation Bar -->
    <div class="navbar">
        <div class="nav-left">
            <span class="nav-logo" onclick="goToHomepage()"><i class="fas fa-briefcase"></i> FresherGuide</span>
        </div>
        <div class="nav-right" id="navRight">
            <div class="user-dropdown" id="userDropdownBtn">
                <div class="user-avatar" id="userAvatar">G</div>
                <span class="user-name" id="displayName">Guest User</span>
                <i class="fas fa-chevron-down"></i>
            </div>
            <div class="dropdown-menu" id="dropdownMenu">
                <a href="#" id="clearProgressBtn"><i class="fas fa-trash-alt"></i> Log Out</a>
            </div>
        </div>
    </div>

    <!-- Scrollable Content Area -->
    <div class="scrollable-content">
        <!-- Modules Grid -->
        <div class="roadmap-grid">
            <!-- 1. Resume Building -->
            <div class="module-card" data-module="Resume Building" data-href="../resume_builder/index.html">
```

CSS CODE

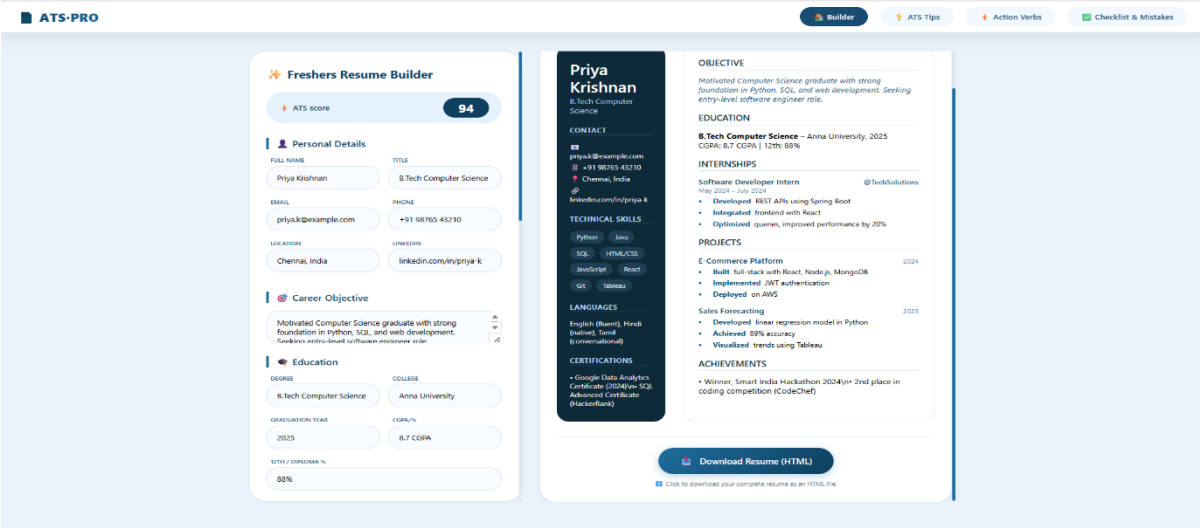
```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

body {
    width: 100%;
    height: 100vh;
    overflow: hidden;
    position: relative;
    background: url('https://images.unsplash.com/photo-1521791136064-7986c2920216?q=80&w=2069&auto=format&fit=crop') no-repeat center center fixed;
    background-size: cover;
}

/* Full Screen Overlay */
body::before {
    content: '';
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.55);
    z-index: -1;
}

.dashboard {
    width: 100%;
    height: 100vh;
    display: flex;
    flex-direction: column;
    position: relative;
    z-index: 2;
    overflow: hidden;
```

JAVASCRIPT CODE

```
dashboard > JS scripts.js > ...
// ========== SHOW TOAST FUNCTION ==========
function showToast(message, type = 'info') {
    const toast = document.createElement('div');
    let className = 'toast-info';
    let icon = '<i class="fas fa-info-circle"></i>';

    if (type === 'success') {
        className = 'toast-success';
        icon = '<i class="fas fa-check-circle"></i>';
    } else if (type === 'warning') {
        className = 'toast-warning';
        icon = '<i class="fas fa-exclamation-triangle"></i>';
    }

    toast.className = 'toast-message ' + className;
    toast.innerHTML = icon + ' ' + message;
    document.body.appendChild(toast);

    setTimeout(() => {
        toast.remove();
    }, 3000);
}

// ========== GO TO HOMEPAGE ==========
function goToHomepage() {
    window.location.href = '../Homepage/index.html';
}

// ========== GET OR CREATE GUEST USER ==========
let guestUser = JSON.parse(localStorage.getItem('fresherguide_guest'));
if (!guestUser) {
    guestUser = { name: 'Guest User', id: 'guest_' + Date.now() };
    localStorage.setItem('fresherguide_guest', JSON.stringify(guestUser));
}

// ========== DOM ELEMENTS ==========
const userDropdownBtn = document.getElementById('userDropdownBtn');
```

6. 1 Resume Building

The image shows a web-based resume builder designed for freshers, featuring a dual-pane interface with a data entry form on the left and a live preview on the right. The application calculates an ATS score of 94 for the current draft, which belongs to a computer science graduate named Priya Krishnan. A prominent blue button at the bottom allows the user to download the completed resume in HTML format.



HTML CODE

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ATS Resume Studio · Clean & Arranged Preview</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<!-- ========== NAVIGATION BAR ========== -->
<nav class="navbar">
    <div class="logo">📄 ATS·PRO</div>
    <div class="nav-links">
        <div class="nav-item active" data-page="resume">🏠 Builder</div>
        <div class="nav-item" data-page="tips">💡 ATS Tips</div>
        <div class="nav-item" data-page="action-verbs">⚡ Action Verbs</div>
        <div class="nav-item" data-page="checklist">✅ Checklist & Mistakes</div>
    </div>
</nav>

<!-- ========= PAGE 1: RESUME BUILDER (with clean arranged preview) ======== -->
<div id="page-resume" class="content-area active">
    <div class="builder-split">
        <!-- LEFT FORM -->
        <div class="builder-left">
            <h2 style="margin-bottom:1.5rem; color:#0f3f5e;">✨ Freshers Resume Builder</h2>
            <div class="score-meter">
                <span style="font-weight:600; color:#0f3f5e;">⚡ ATS score</span>
                <span class="score-value" id="scoreMeter">88</span>
            </div>

            <!-- Personal Details -->
            <div class="section-head">👤 Personal Details</div>
            <div class="grid-2">
                <div class="input-group"><label>Full name</label><input type="text" id="nameInput" value="Priya Krishnan"></div>
                <div class="input-group"><label>Title</label><input type="text" id="titleInput" value="B.Tech Computer Science"></div>
```

CSS CODE

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

body {
    background: #e9f1fa;
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* ---------- NAVIGATION BAR ---------- */
.navbar {
    width: 100%;
    background: white;
    box-shadow: 0 8px 25px rgba(0,20,40,0.08);
    padding: 0.9rem 2.5rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    border-bottom: 1px solid #c0d9f0;
    position: sticky;
    top: 0;
    z-index: 1000;
}
.logo {
    font-weight: 800;
    font-size: 1.6rem;
    background: linear-gradient(145deg, #0b3b5a, #1f6e9e);
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
```

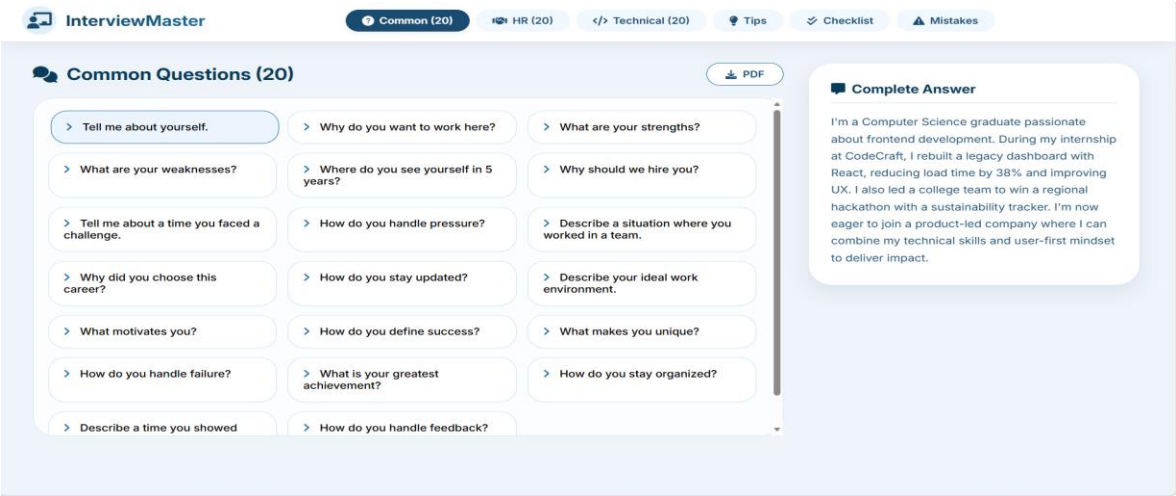
JAVASCRIPT CODE

```
resume builder > JS scriptjs > ...
(function() {
    // navigation
    const navItems = document.querySelectorAll('.nav-item');
    const pages = document.querySelectorAll('.content-area');
    navItems.forEach(item => {
        item.addEventListener('click', function() {
            const pageId = this.dataset.page;
            navItems.forEach(n => n.classList.remove('active'));
            this.classList.add('active');
            pages.forEach(p => p.classList.rem  const pageId: any
            document.getElementById('page-' + pageId).classList.add('active');
        });
    });

    // data stores
    let internships = [
        { title: 'Software Developer Intern', company: 'TechSolutions', date: 'May 2024 - July 2024', desc: 'Developed REST APIs using Spring Boot;Integrated frontend wit
    ];
    let projects = [
        { title: 'E-Commerce Platform', date: '2024', desc: 'Built full-stack with React, Node.js, MongoDB;Implemented JWT authentication;Deployed on AWS' },
        { title: 'Sales Forecasting', date: '2023', desc: 'Developed linear regression model in Python;Achieved 89% accuracy;Visualized trends using Tableau' }
    ];

    const internsContainer = document.getElementById('internsContainer');
    const projectsContainer = document.getElementById('projectsContainer');

    function renderInterns() {
        let html = '';
        internships.forEach((inv, idx) => {
            html += `<div class="entry-card">
                <div><strong>Internship ${idx+1}</strong><button class="remove-btn" data-type="intern" data-index="${idx}">✕ Remove</button></div>
                <div class="grid-2"><div class="input-group"><label>Title</label><input value="${inv.title.replace(/"/g, '&quot;')}" class="intern-title" data-index="${id
                <div class="input-group"><label>Date</label><input value="${inv.date}" class="intern-date" data-index="${idx}"></div>
                <div class="input-group"><label>Bullet points (;)</label><textarea rows="2" class="intern-desc" data-index="${idx}">${inv.desc}</textarea></div>
            </div>`;
        });
```

6.2 Interview Preparation

The InterviewMaster dashboard is an interview preparation tool that categorizes practice questions into Common, HR, and Technical groups.

It provides an interactive list of standard prompts (like "Tell me about yourself") alongside a "Complete Answer" sidebar that offers structured sample responses. The interface also includes navigation for interview tips, checklists, and common mistakes to help candidates prepare.



HTML CODE

```html
interview > index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Interview </title>
    <!-- Google Font & Font Awesome -->
    <link rel="preconnect"
    <link rel="preconnect"
    <link href="https://fon                                                    2,600;14..32,700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <!-- jsPDF -->
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<nav class="navbar">
    <div class="logo-area">
        <div class="logo-icon"><i class="fas fa-chalkboard-user"></i></div>
        <span class="logo-text">Interview</span>
    </div>
    <div class="nav-links">
        <div class="nav-option active" data-mode="common"><i class="fas fa-question-circle"></i> Common</div>
        <div class="nav-option" data-mode="hr"><i class="fas fa-handshake"></i> HR</div>
        <div class="nav-option" data-mode="technical"><i class="fas fa-code"></i> Technical</div>
        <div class="nav-option" data-mode="tips"><i class="fas fa-lightbulb"></i> Tips</div>
        <div class="nav-option" data-mode="checklist"><i class="fas fa-check-double"></i> Checklist</div>
        <div class="nav-option" data-mode="mistakes"><i class="fas fa-exclamation-triangle"></i> Mistakes</div>
    </div>
    <div style="width: 20px;"></div>
</nav>
<div class="main-wrapper">
    <!-- COMMON SECTION -->
    <div id="commonSection" class="content-section active">
        <div class="two-column">
            <div class="left-col">
                <div class="category-header">
```

This attribute specifies the URL of the linked resource. A URL can be absolute or relative.

Widely available across major browsers (Baseline since 2015)

CSS CODE :

```css
interview > style.css > *
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}
body {
    background: #eef4fa;
}
/* ========= TOP NAVIGATION ========= */
.navbar {
    background: white;
    border-bottom: 1px solid #cbdbe6;
    padding: 0.7rem 2rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.05);
    position: sticky;
    top: 0;
    z-index: 1000;
}
.logo-area {
    display: flex;
    align-items: center;
    gap: 12px;
}
.logo-icon {
    background: #e3eff9;
    width: 45px;
    height: 45px;
    border-radius: 40px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.6rem;
```

JAVASCRIPT CODE:

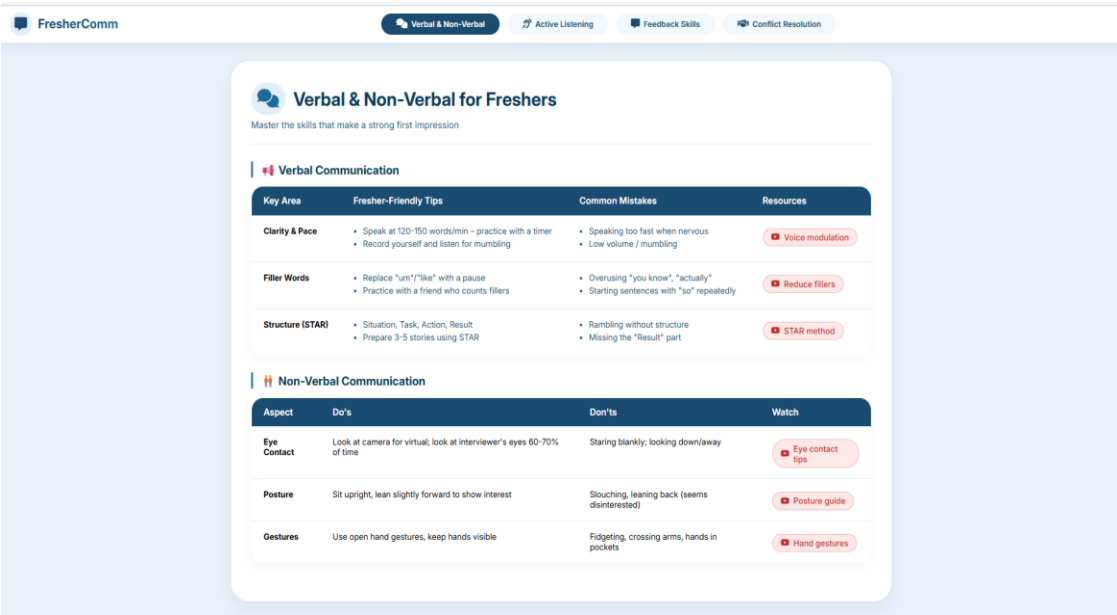
```javascript
(function() {
    const technicalQuestions = [
    ];
    // ensure exactly 20 tech
    while(technicalQuestions.length < 20) technicalQuestions.push({ q: "What is a deadlock?", a: "Deadlock occurs when processes wait indefinitely for resources. Conditio
    function renderGrid(containerId, data, category) {
        const grid = document.getElementById(containerId);
        if (!grid) return;
        grid.innerHTML = data.map((item, idx) => `
            <div class="q-item" data-category="${category}" data-qindex="${idx}">
                <i class="fas fa-angle-right"></i> ${item.q}
            </div>
        `).join('');
    }
    renderGrid('commonGrid', commonQuestions, 'common');
    renderGrid('hrGrid', hrQuestions, 'hr');
    renderGrid('technicalGrid', technicalQuestions, 'technical');
    // answer panels
    const commonAns = document.getElementById('commonAnswerText');
    const hrAns = document.getElementById('hrAnswerText');
    const techAns = document.getElementById('technicalAnswerText');
    function attachListeners() {
        document.querySelectorAll('.q-item').forEach(el => {
            el.removeEventListener('click', clickHandler);
            el.addEventListener('click', clickHandler);
        });
    }
    function clickHandler(e) {
        const cat = this.dataset.category;
        const idx = parseInt(this.dataset.qindex);
        if (cat === 'common' && commonQuestions[idx]) commonAns.innerText = commonQuestions[idx].a;
        else if (cat === 'hr' && hrQuestions[idx]) hrAns.innerText = hrQuestions[idx].a;
        else if (cat === 'technical' && technicalQuestions[idx]) techAns.innerText = technicalQuestions[idx].a;
    }
    attachListeners();
    // PDF generation
    function generatePDF(title, questionsArr) {
```

6.3 Communication

The page is a communication guide for new professionals (freshers) that outlines two main areas:

Verbal Communication: Focuses on clarity, pace, avoiding filler words, and using the STAR method (Situation, Task, Action, Result) for structured speaking.

Non-Verbal Communication: Covers eye contact (60-70% of the time), maintaining an upright posture, and using open hand gestures.



HTML CODE

```html
communication > index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fresher Communication · Table Format</title>
    <!-- Google Font & Font Awesome -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,400;14..32,500;14..32,600;14..32,700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- ========== TOP NAVIGATION BAR ========== -->
    <nav class="navbar">
        <div class="logo-area">
            <div class="logo-icon"><i class="fas fa-message"></i></div>
            <span class="logo-text">FresherComm</span>
        </div>
        <div class="nav-links-wrapper">
            <div class="nav-links">
                <div class="nav-option active" data-mode="verbal"><i class="fas fa-comments"></i> Verbal & Non-Verbal</div>
                <div class="nav-option" data-mode="listening"><i class="fas fa-ear-listen"></i> Active Listening</div>
                <div class="nav-option" data-mode="feedback"><i class="fas fa-message"></i> Feedback Skills</div>
                <div class="nav-option" data-mode="conflict"><i class="fas fa-handshake"></i> Conflict Resolution</div>
            </div>
        </div>
        <div style="width: 60px;"></div>
    </nav>
    <!-- MAIN CONTENT -->
    <div class="main-wrapper">
        <!-- ========== VERBAL & NON-VERBAL PAGE ========== -->
        <div id="verbalSection" class="content-section active">
            <div class="comm-page">
                <div class="page-header">
                    <h1><i class="fas fa-comments"></i> Verbal & Non-Verbal for Freshers</h1>
```

CSS CODE

```css
communication > style.css > *
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}
body {
    background: #e9f1fa;
}
/* ========== TOP NAVIGATION BAR ========== */
.navbar {
    background: white;
    border-bottom: 1px solid #c0d9f0;
    padding: 0.8rem 2rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    box-shadow: 0 8px 20px rgba(0, 20, 40, 0.05);
    position: sticky;
    top: 0;
    z-index: 1000;
}
.logo-area {
    display: flex;
    align-items: center;
    gap: 12px;
    margin-right: 2rem;
}
.logo-icon {
    background: #e2effa;
    width: 45px;
    height: 45px;
    border-radius: 40px;
    display: flex;
    align-items: center;
    justify-content: center;
```

JAVASCRIPT CODE

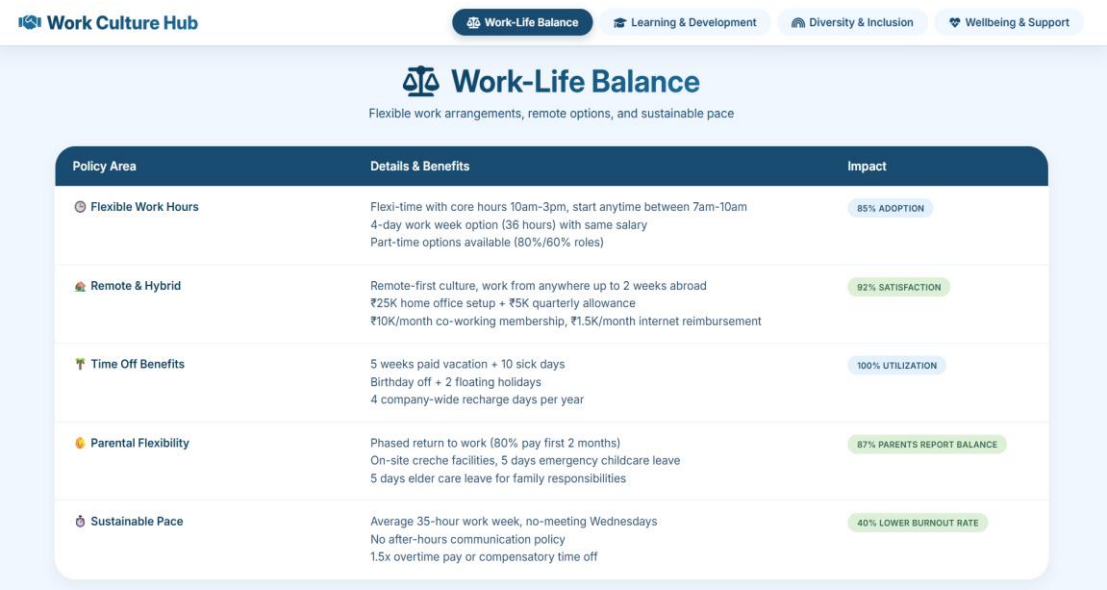
```
communication > JS scripts.js > ...
(function() {
    // Tab switching functionality
    const navOptions = document.querySelectorAll('.nav-option');
    const sections = {
        verbal: document.getElementById('verbalSection'),
        listening: document.getElementById('listeningSection'),
        feedback: document.getElementById('feedbackSection'),
        conflict: document.getElementById('conflictSection')
    };

    navOptions.forEach(opt => {
        opt.addEventListener('click', () => {
            navOptions.forEach(o => o.classList.remove('active'));
            opt.classList.add('active');
            const mode = opt.dataset.mode;
            Object.values(sections).forEach(s => s?.classList.remove('active'));
            if (sections[mode]) sections[mode].classList.add('active');
            window.scrollTo({ top: 0, behavior: 'smooth' });
        });
    });

    const scrollBtn = document.getElementById('scrollBtn');
    window.addEventListener('scroll', () => {
        if (window.scrollY > 300) {
            scrollBtn.style.display = 'block';
        } else {
            scrollBtn.style.display = 'none';
        }
    });
    scrollBtn.addEventListener('click', () => {
        window.scrollTo({ top: 0, behavior: 'smooth' });
    });
    const videoLinks = document.querySelectorAll('.youtube-link');
    videoLinks.forEach(link => {
        link.addEventListener('click', (e) => {
            e.preventDefault();
            const videoTopic = link.getAttribute('data-video') || link.textContent.trim();
```

6. 4 WorkCulture

This page outlines Work-Life Balance policies, featuring a remote-first culture with core hours (10am–3pm) and an optional 4-day work week.

It offers extensive benefits including 5 weeks of paid vacation, 4 annual recharge days, and financial support for home office setups.

The initiative aims for a sustainable pace through 35-hour weeks and "no-meeting Wednesdays," resulting in a reported 40% lower burnout rate.



HTML CODE

```html
workculture > index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Work Culture · Complete Guide for Freshers</title>
    <!-- Google Font & Font Awesome -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,400;14..32,500;14..32,600;14..32,700;14..32,800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <!-- External CSS -->
    <link rel="stylesheet" href="style.css">
</head>
<body>
<!-- ========== TOP NAVIGATION BAR ========== -->
<nav class="navbar">
    <div class="logo">
        <i class="fas fa-handshake"></i>
        <span>Work Culture Hub</span>
    </div>
    <div class="nav-links">
        <div class="nav-item active" data-page="worklife">
            <i class="fas fa-balance-scale"></i> Work-Life Balance
        </div>
        <div class="nav-item" data-page="learning">
            <i class="fas fa-graduation-cap"></i> Learning & Development
        </div>
        <div class="nav-item" data-page="diversity">
            <i class="fas fa-rainbow"></i> Diversity & Inclusion
        </div>
        <div class="nav-item" data-page="wellbeing">
            <i class="fas fa-heartbeat"></i> Wellbeing & Support
        </div>
    </div>
</nav>
<!-- ========== MAIN CONTENT ========== -->
```

CSS CODE

```css
workculture > style.css > ...
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

body {
    background: #f0f7ff;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding-top: 80px;
}

/* ========== TOP NAVIGATION BAR ========== */
.navbar {
    width: 100%;
    background: white;
    box-shadow: 0 8px 25px rgba(0, 20, 40, 0.08);
    padding: 0.8rem 2rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    border-bottom: 1px solid #c0d9f0;
    position: fixed;
    top: 0;
    z-index: 1000;
}

.logo {
    font-weight: 800;
    font-size: 1.5rem;
    display: flex;
    align-items: center;
    gap: 8px;
```

JAVASCRIPT CODE

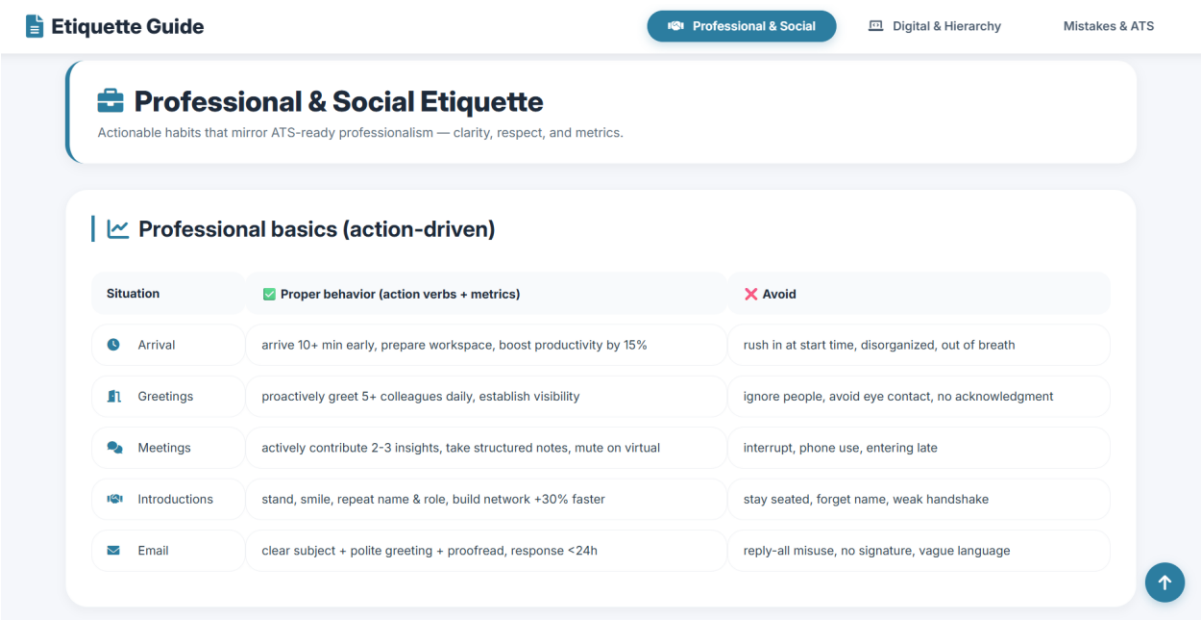
```javascript
// ========== TOP NAVIGATION CONTROLLER ==========
// Get all navigation items and page sections
const navItems = document.querySelectorAll('.nav-item');
const pageSections = document.querySelectorAll('.page-section');

/**
 * Switch between different content pages
 * @param {string} pageId - The identifier for the page to display (worklife, learning, diversity, wellbeing)
 */
function switchPage(pageId) {
    pageSections.forEach(section => {
        section.classList.remove('active-section');
    });
    const activePage = document.getElementById('page-' + pageId);
    if (activePage) {
        activePage.classList.add('active-section');
    }
    navItems.forEach(item => {
        item.classList.remove('active');
    });

    const activeNav = document.querySelector(`.nav-item[data-page="${pageId}"]`);
    if (activeNav) {
        activeNav.classList.add('active');
    }
        window.scrollTo({
        top: 0,
        behavior: 'smooth'
    });
}

navItems.forEach(item => {
    item.addEventListener('click', function() {
        const pageId = this.getAttribute('data-page');
        if (pageId) {
            switchPage(pageId);
        }
```

6.5 Office Etiquette

Measurable Professionalism: It converts "soft skills" into quantifiable metrics (e.g., "5+ colleagues daily," "10+ min early") to track and boost personal productivity.

Behavioral Optimization: It contrasts high-impact actions (like active meeting contributions) against common pitfalls (like phone use) to establish a clear standard for workplace conduct.

ATS-Ready Identity: it frames daily behavior as a data-driven extension of a resume, ensuring personal interactions are as clear and results-oriented as a professional profile.



Situation	Proper behavior (action verbs + metrics)	Avoid
Arrival	arrive 10+ min early, prepare workspace, boost productivity by 15%	rush in at start time, disorganized, out of breath
Greetings	proactively greet 5+ colleagues daily, establish visibility	ignore people, avoid eye contact, no acknowledgment
Meetings	actively contribute 2-3 insights, take structured notes, mute on virtual	interrupt, phone use, entering late
Introductions	stand, smile, repeat name & role, build network +30% faster	stay seated, forget name, weak handshake
Email	clear subject + polite greeting + proofread, response <24h	reply-all misuse, no signature, vague language

HTML CODE

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title> Office Etiquette</title>
    <!-- Google Font & Font Awesome -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,400;14..32,500;14..32,600;14..32,700;14..32,800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- ========== NAVIGATION (WHITE + BLUE ACTIVE) ========== -->
    <nav class="navbar">
        <div class="logo">
            <i class="fas fa-file-alt"></i>
            <span> Etiquette Guide</span>
        </div>
        <div class="nav-links">
            <div class="nav-item active" data-block="proSocialBlock">
                <i class="fas fa-handshake"></i> Professional & Social
            </div>
            <div class="nav-item" data-block="digitalHierBlock">
                <i class="fas fa-laptop-code"></i> Digital & Hierarchy
            </div>
            <div class="nav-item" data-block="mistakeBlock">
                <i class="fas fa-chart-simple"></i> Mistakes & ATS
            </div>
        </div>
    </nav>
    <main class="main-content">
        <!-- ========== SECTION 1: PROFESSIONAL & SOCIAL ETIQUETTE ========== -->
        <div id="proSocialBlock" class="section-block active-block">
            <div class="page-header">
                <h1><i class="fas fa-briefcase"></i> Professional & Social Etiquette</h1>
```

CSS CODE

```
tiquette > # style.css > ...
/* ========== GLOBAL RESET & BASE ========== */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}

body {
    background: #f5f7fb;
    min-height: 100vh;
    padding-top: 85px;
}

/* ========== NAVIGATION (white bg, blue active) ========== */
.navbar {
    width: 100%;
    background: #ffffff;
    box-shadow: 0 2px 12px rgba(0, 0, 0, 0.08);
    padding: 0.9rem 2.5rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    border-bottom: 1px solid #e9ecef;
    position: fixed;
    top: 0;
    z-index: 1000;
}
.logo {
    font-weight: 800;
    font-size: 1.6rem;
    color: #1e2a3a;
    display: flex;
    align-items: center;
    gap: 10px;
}
```

JAVASCRIPT CODE

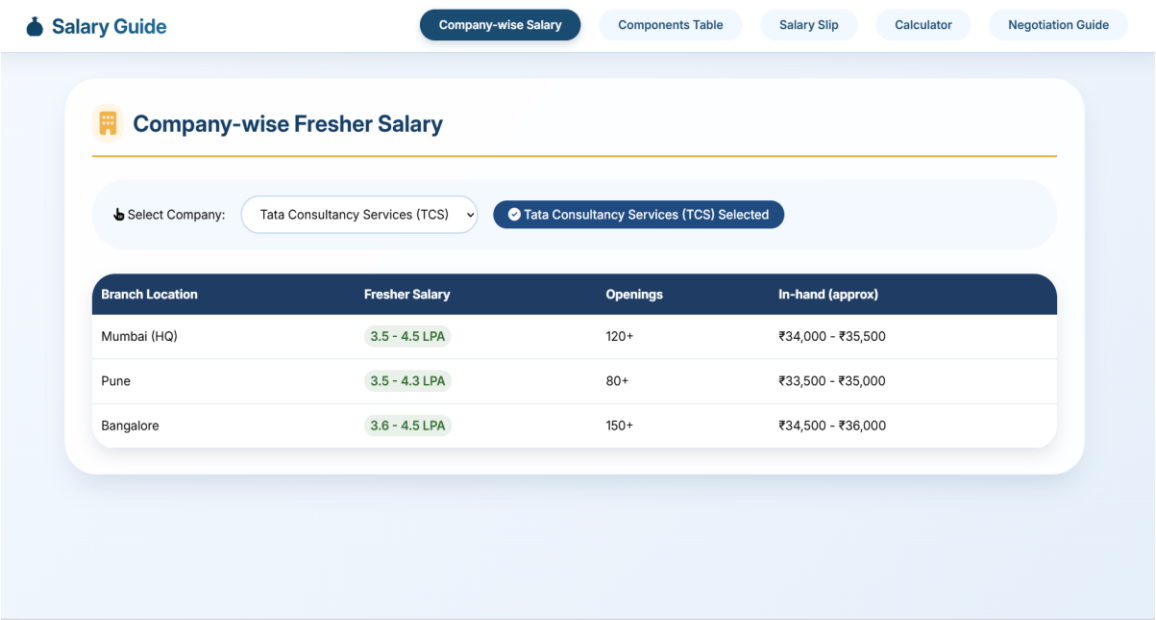
```
ts > etiquette > JS script.js > ...
(function() {
    // DOM elements
    const navItems = document.querySelectorAll('.nav-item');
    const blocks = {
        proSocialBlock: document.getElementById('proSocialBlock'),
        digitalHierBlock: document.getElementById('digitalHierBlock'),
        mistakeBlock: document.getElementById('mistakeBlock')
    };

    // Helper: deactivate all blocks & nav active classes
    function deactivateAll() {
        Object.values(blocks).forEach(block => {
            if (block) block.classList.remove('active-block');
        });
        navItems.forEach(btn => btn.classList.remove('active'));
    }
        function switchToBlock(blockId) {
        if (!blockId || !blocks[blockId]) return;

        deactivateAll();
        blocks[blockId].classList.add('active-block');

        const targetNav = Array.from(navItems).find(btn => btn.dataset.block === blockId);
        if (targetNav) targetNav.classList.add('active');
        const mainDiv = document.querySelector('.main-content');
        if (mainDiv) {
            mainDiv.scrollTo({ top: 0, behavior: 'smooth' });
        } else {
            window.scrollTo({ top: 0, behavior: 'smooth' });
        }
    }

    navItems.forEach(btn => {
        btn.addEventListener('click', function(e) {
            e.preventDefault();
            const blockId = this.dataset.block;
            if (blockId && blocks[blockId]) {
```

6.6 Salary Module

The page provides a fresher salary breakdown for Tata Consultancy Services (TCS) across Mumbai, Pune, and Bangalore for 2026.

It lists key financial metrics including annual CTC (LPA), the number of active openings, and estimated monthly in-hand pay.

The data serves as a comparative guide to help job seekers understand how compensation varies by office location.



HTML CODE

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Salary Calculator · CTC to Take-Home</title>
    <!-- Google Font -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:opsz,wght@14..32,400;14..32,500;14..32,600;14..32,700&display=swap" rel="stylesheet">
    <!-- Font Awesome -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- ===== NAVIGATION BAR ===== -->
    <nav class="navbar">
        <div class="logo">💰 Salary Guide</div>
        <div class="nav-links">
            <div class="nav-item active" data-target="company-section">Company-wise Salary</div>
            <div class="nav-item" data-target="components-section">Components Table</div>
            <div class="nav-item" data-target="slip-section">Salary Slip</div>
            <div class="nav-item" data-target="calculator-section">Calculator</div>
            <div class="nav-item" data-target="negotiation-section">Negotiation Guide</div>
        </div>
    </nav>
    <!-- ===== MAIN CONTENT ===== -->
    <div class="main-content">
        <!-- COMPANY SECTION -->
        <section id="company-section" class="section">
            <div class="section-card">
                <div class="section-title"><i class="fas fa-building"></i> Company-wise Fresher Salary</div>
                <div class="company-selector">
                    <span class="selector-label"><i class="fas fa-hand-pointer"></i> Select Company:</span>
                    <select id="companySelect">
                        <option value="tcs">Tata Consultancy Services (TCS)</option>
                        <option value="infosys">Infosys</option>
```

CSS CODE

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Inter', sans-serif;
}

body {
    min-height: 100vh;
    background: linear-gradient(135deg, #f5f9ff 0%, #e6f0fa 100%);
    padding-top: 80px;
}
/* ===== NAVIGATION BAR ===== */
.navbar {
    width: 100%;
    background: white;
    box-shadow: 0 8px 25px rgba(0, 20, 40, 0.08);
    padding: 0.9rem 2.5rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    border-bottom: 1px solid #c0d9f0;
    position: fixed;
    top: 0;
    z-index: 1000;
    backdrop-filter: blur(4px);
}
.logo {
    font-weight: 800;
    font-size: 1.6rem;
    background: linear-gradient(145deg, #0b3b5a, #1f6e9e);
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
    letter-spacing: -0.3px;
```

JAVASCRIPT CODE

```javascript
(function() {
    // Navigation
    const navItems = document.querySelectorAll('.nav-item');
    const sections = {
        'company-section': document.getElementById('company-section'),
        'components-section': document.getElementById('components-section'),
        'slip-section': document.getElementById('slip-section'),
        'calculator-section': document.getElementById('calculator-section'),
        'negotiation-section': document.getElementById('negotiation-section')
    };
    function showSection(targetId) {
        Object.values(sections).forEach(section => {
            if (section) section.classList.add('hidden');
        });
        if (sections[targetId]) sections[targetId].classList.remove('hidden');
        navItems.forEach(item => item.classList.remove('active'));
        const activeItem = document.querySelector(`.nav-item[data-target="${targetId}"]`);
        if (activeItem) activeItem.classList.add('active');
    }
    navItems.forEach(item => {
        item.addEventListener('click', function(e) {
            e.preventDefault();
            const target = this.getAttribute('data-target');
            showSection(target);
            window.scrollTo({ top: 0, behavior: 'smooth' });
        });
    });
    showSection('company-section');

    // Company data
    const companyBranches = {
        tcs: { name: 'Tata Consultancy Services (TCS)', branches: [
            { location: 'Mumbai (HQ)', fresherSalary: '3.5 - 4.5 LPA', openings: '120+', inhand: '₹34,000 - ₹35,500' },
            { location: 'Pune', fresherSalary: '3.5 - 4.3 LPA', openings: '80+', inhand: '₹33,500 - ₹35,000' },
            { location: 'Bangalore', fresherSalary: '3.6 - 4.5 LPA', openings: '150+', inhand: '₹34,500 - ₹36,000' }
        ]},
        infosys: { name: 'Infosys', branches: [
```

7. Database Implementation

Introduction

The database setup is used to store and manage user registration and login data in the system. In the First-Time Job Guidance for Freshers project, MySQL is used as the database, and Spring Boot is used to connect the backend with the database.

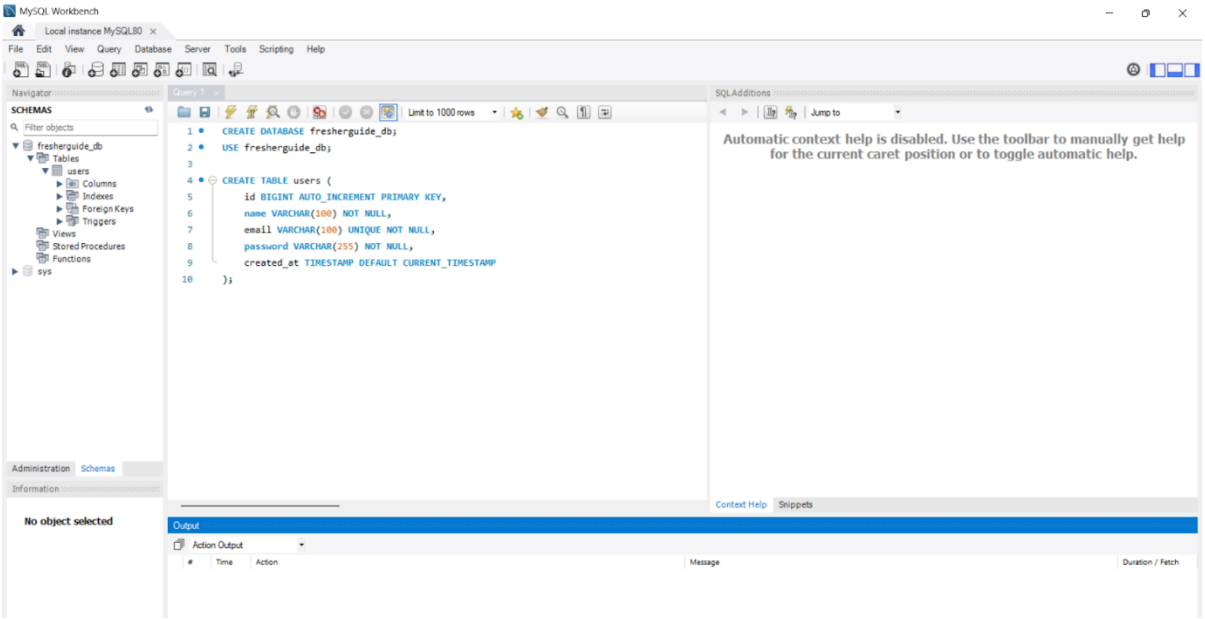
This setup ensures that user data is stored securely and can be accessed during login for authentication.

Software Requirements

- MySQL Server
- MySQL Workbench
- Java (JDK)
- Spring Boot
- IntelliJ IDEA

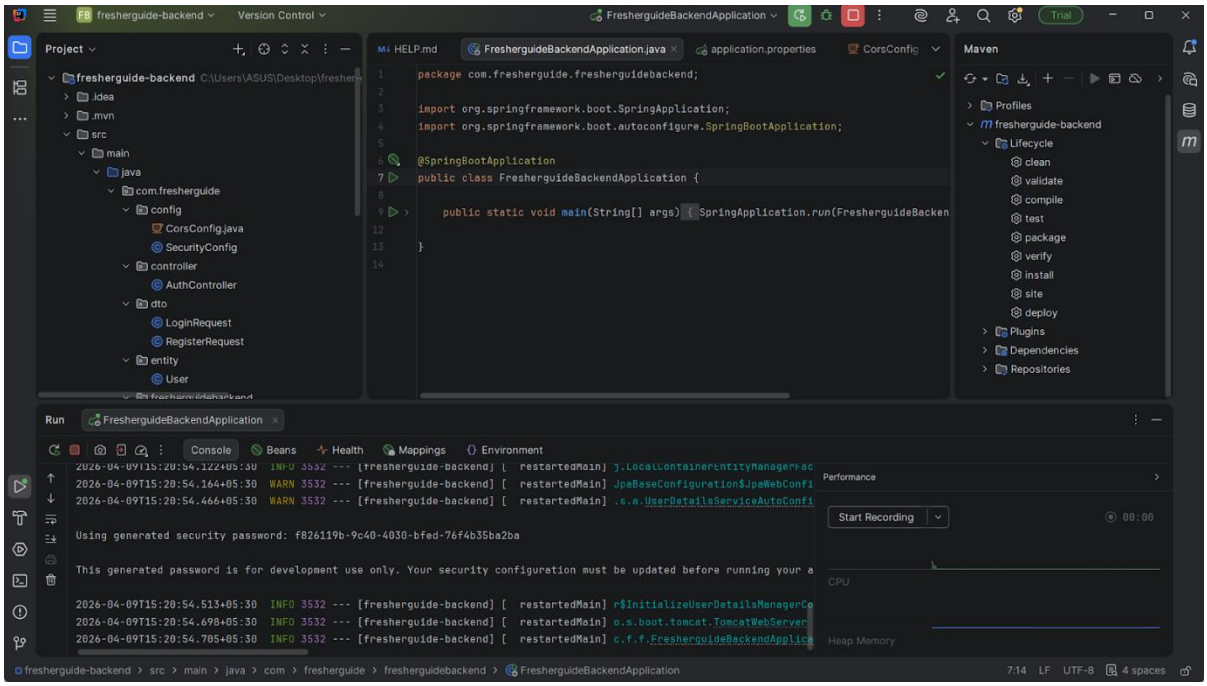
7.1 MySQL Database

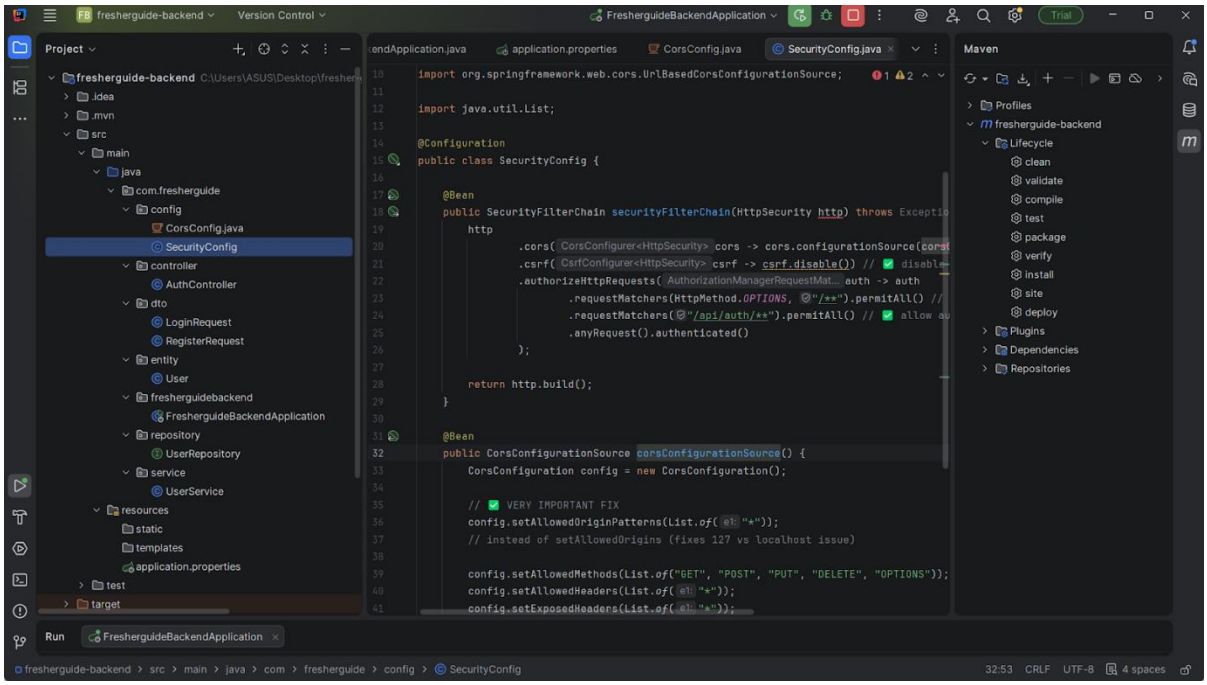
MySQL database is used in the “Freshers Guide” project to store, manage, and organize all the application data in a structured manner. It stores important user information such as registration details, profile data, and login credentials. It is also used to manage resume builder data including personal information, education details, skills, and work experience. In addition, MySQL stores interview preparation content such as common, HR, and technical questions along with their model answers. The database also handles salary-related information like CTC structure, salary breakdown, company-wise salary details, and negotiation guidance. All data is stored in the form of tables, which makes it easy to retrieve, update, and manage information efficiently. MySQL supports relationships between different tables using primary and foreign keys, ensuring data consistency and integrity. It also helps in performing CRUD operations (Create, Read, Update, Delete) required for different modules of the system. Overall, MySQL plays a crucial role in providing a reliable, secure, and scalable backend for the “Freshers Guide” project.

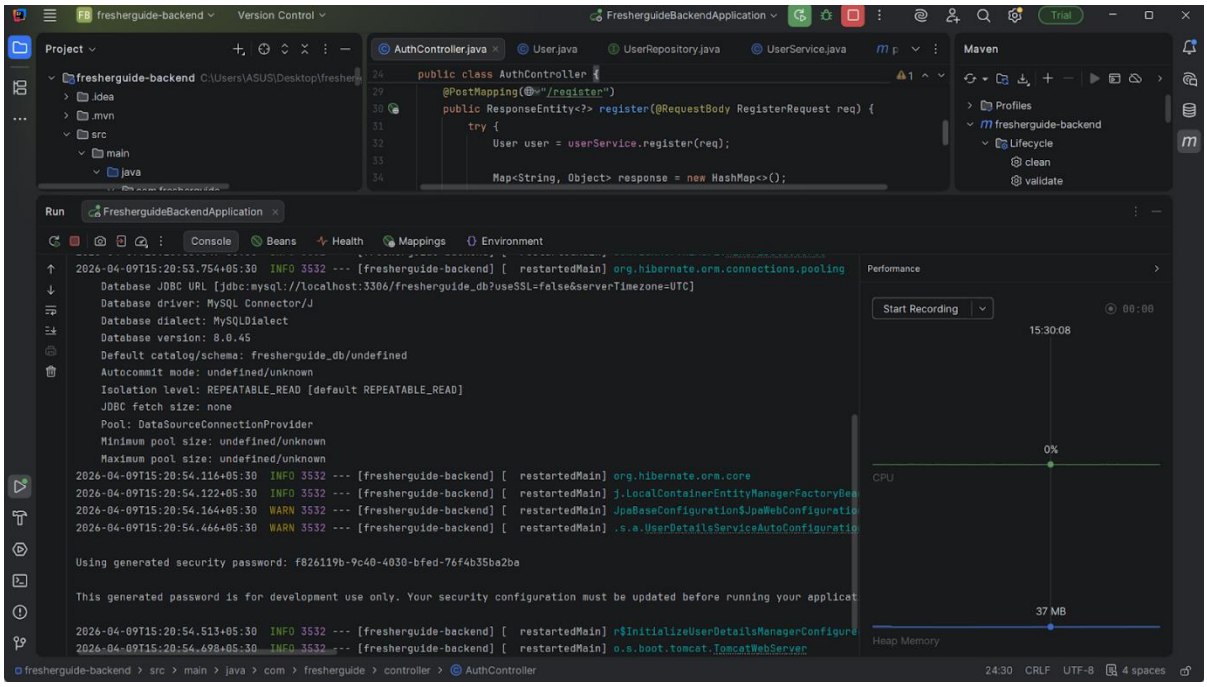


7.2 Backend Server

The backend server in the “Freshers Guide” project is responsible for handling all the core logic, processing user requests, and managing communication between the frontend and the database. It is developed using Spring Boot, which provides a fast, secure, and scalable environment for building web applications. The backend server receives requests from the user interface, processes them, and interacts with the MySQL database to store or retrieve data such as user details, resume information, interview questions, and salary data. It also handles important functions like authentication, data validation, and CRUD operations (Create, Read, Update, Delete). The backend ensures that all modules of the system work smoothly and securely by maintaining proper data flow and business logic. Overall, the backend server acts as the backbone of the “Freshers Guide” project, ensuring efficient performance and reliable data management.

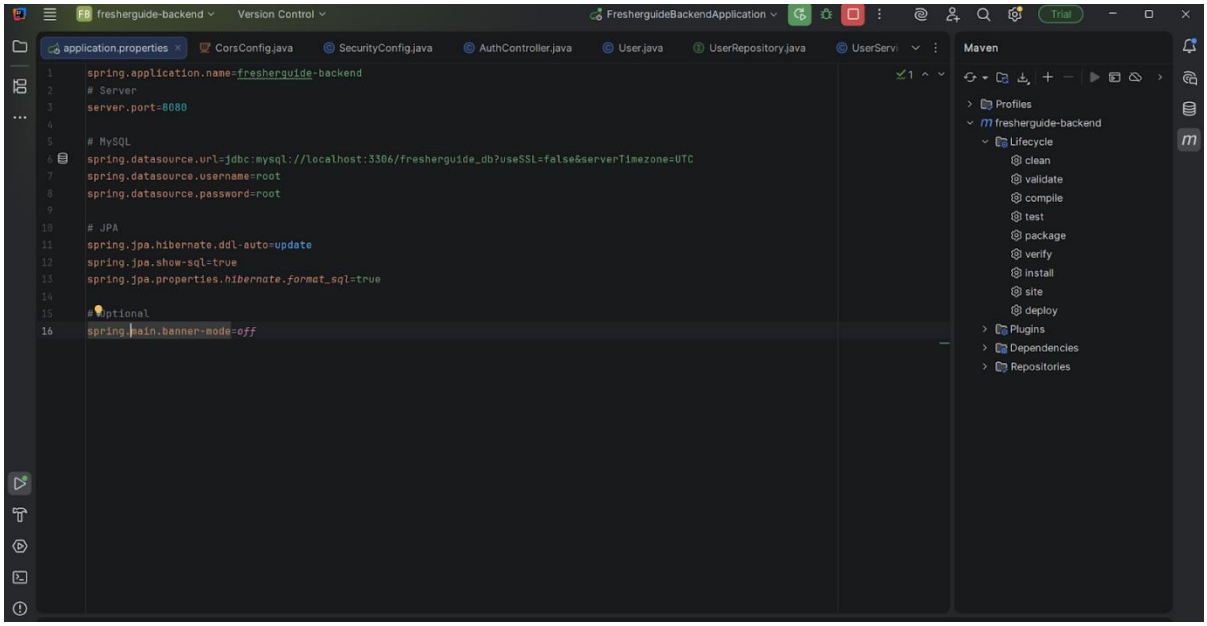



fresherguide-backend
Version Control
FresherguideBackendApplication
Trial
Project
fresherguide-backend C:\Users\ASUS\Desktop\fresher
.idea
.mvn
src
main
java
com.fresherguide
config
CorsConfig.java
SecurityConfig
controller
AuthController
dto
LoginRequest
RegisterRequest
entity
User
fresherguidebackend
FresherguideBackendApplication
repository
UserRepository
service
UserService
resources
static
templates
application.properties
test
target
endApplication.java
application.properties
CorsConfig.java
SecurityConfig.java
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;
@Configuration
public class SecurityConfig {
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exceptio
http
.cors(CorsConfigurer<HttpSecurity> cors -> cors.configurationSource(cors
.csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf.disable()) // disable
.authorizeHttpRequests(AuthorizationManagerRequestMat... auth -> auth
.requestMatchers(HttpMethod.OPTIONS, "/**").permitAll() //
.requestMatchers("/api/auth/**").permitAll() // allow au
.anyRequest().authenticated()
);
return http.build();
}
@Bean
public CorsConfigurationSource corsConfigurationSource() {
CorsConfiguration config = new CorsConfiguration();
// VERY IMPORTANT FIX
config.setAllowedOriginPatterns(List.of(e1: "*"));
// instead of setAllowedOrigins (fixes 127 vs localhost issue)
config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
config.setAllowedHeaders(List.of(e1: "*"));
config.setExposedHeaders(List.of(e1: "*"));
Maven
Profiles
fresherguide-backend
Lifecycle
clean
validate
compile
test
package
verify
install
site
deploy
Plugins
Dependencies
Repositories
Run
FresherguideBackendApplication
fresherguide-backend > src > main > java > com > fresherguide > config > SecurityConfig
32:53
CRLF
UTF-8
4 spaces


fresherguide-backend
Version Control
FresherguideBackendApplication
Trial
Project
fresherguide-backend C:\Users\ASUS\Desktop\fresher
.idea
.mvn
src
main
java
AuthController.java
User.java
UserRepository.java
UserService.java
public class AuthController {
@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest req) {
try {
User user = userService.register(req);
Map<String, Object> response = new HashMap<>();
Maven
Profiles
fresherguide-backend
Lifecycle
clean
validate
Run
FresherguideBackendApplication
Console
Beans
Health
Mappings
Environment
2026-04-09T15:20:53.754+05:30 INFO 3532 --- [fresherguide-backend] [restartedMain] org.hibernate.orm.connections.pooling
Database JDBC URL [jdbc:mysql://localhost:3306/fresherguide_db?useSSL=false&serverTimezone=UTC]
Database driver: MySQL Connector/J
Database dialect: MySQLDialect
Database version: 8.0.45
Default catalog/schema: fresherguide_db/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-04-09T15:20:54.116+05:30 INFO 3532 --- [fresherguide-backend] [restartedMain] org.hibernate.orm.core
2026-04-09T15:20:54.122+05:30 INFO 3532 --- [fresherguide-backend] [restartedMain] j.LocalContainerEntityManagerFactoryBea
2026-04-09T15:20:54.164+05:30 WARN 3532 --- [fresherguide-backend] [restartedMain] JpaBaseConfiguration$JpaWebConfiguratio
2026-04-09T15:20:54.466+05:30 WARN 3532 --- [fresherguide-backend] [restartedMain] .s.a.UserDetailsServiceAutoConfiguratio
Using generated security password: f826119b-9c40-4030-bfed-76f4b35ba2ba
This generated password is for development use only. Your security configuration must be updated before running your applicat
2026-04-09T15:20:54.513+05:30 INFO 3532 --- [fresherguide-backend] [restartedMain] r$InitializeUserDetailsManagerConfigure
2026-04-09T15:20:54.698+05:30 INFO 3532 --- [fresherguide-backend] [restartedMain] o.s.boot.tomcat.TomcatWebServer
Performance
Start Recording
00:00
15:30:08
0%
CPU
37 MB
Heap Memory
fresherguide-backend > src > main > java > com > fresherguide > controller > AuthController
24:30
CRLF
UTF-8
4 spaces

7.3 Database Connection

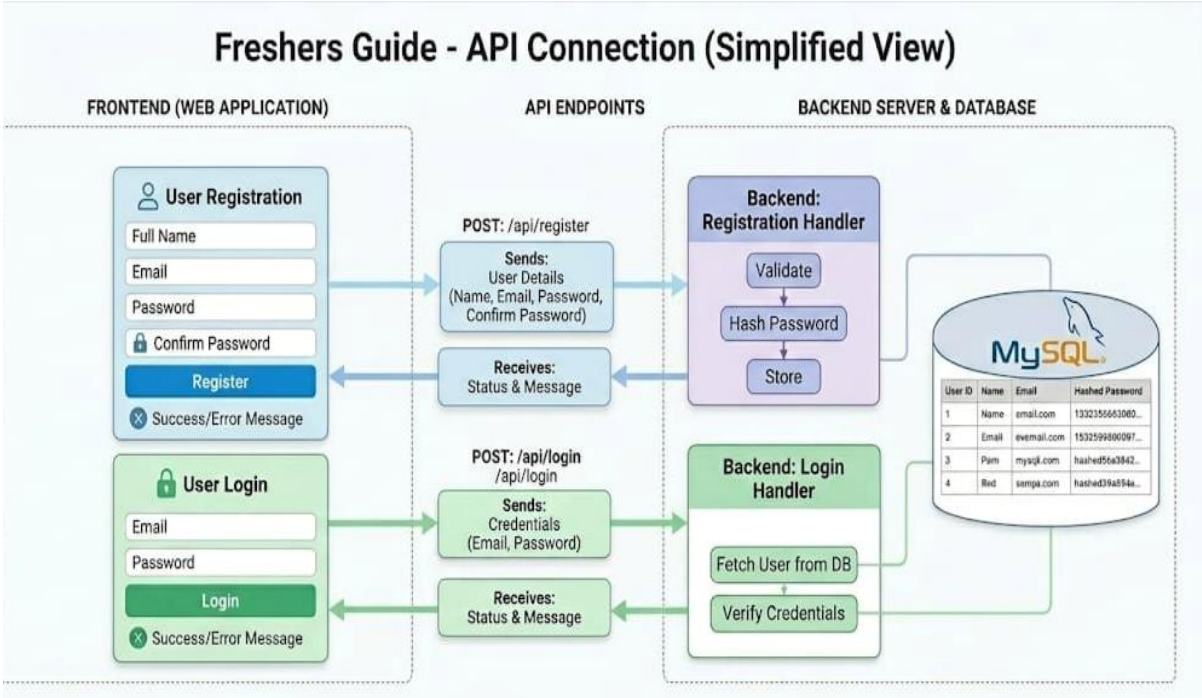
The database connection in the “Freshers Guide” project is used to establish communication between the backend server and the MySQL database. It allows the application to store, retrieve, update, and delete data efficiently. In this project, Spring Boot is used to connect with MySQL using JDBC (Java Database Connectivity) and Spring Data JPA. The connection is configured in the application properties file, where details such as database URL, username, password, and driver class are defined. Once the connection is established, the backend can perform CRUD operations on different tables like user data, resume details, interview questions, and salary information. The database connection ensures smooth data flow between the frontend, backend, and database, making the system reliable and efficient. It also helps maintain data security and consistency throughout the application.



8. API IMPLEMENTATION

REST (Representational State Transfer) APIs are used in this project to enable communication between the frontend and backend. The frontend sends HTTP requests, and the backend processes them and returns responses in JSON format.

In the **First-Time Job Guidance for Freshers** project, REST APIs are used to manage user data, resumes, interview questions, and contact queries efficiently.



8.1 GET API

Purpose:

The GET API is used to retrieve user data from the database after successful login. It allows users to view their stored information without making any changes. This API plays an important role in displaying personalized content to each user based on their login credentials. It ensures secure and controlled access to user-specific data.

Used in Project:

- Display logged-in user information on the dashboard
- Show profile details such as name, email, and contact information

Working:

After a successful login, the frontend sends a GET request to the backend server. The request includes user identification details such as user ID or authentication token. The backend receives this request and validates the user session. After validation, it connects to the MySQL database and retrieves the required user data from relevant tables. The retrieved data is then converted into JSON format and sent back to the frontend. Finally, the frontend displays this data on the user dashboard in a structured and user-friendly manner.

This process ensures that data is fetched efficiently without modifying any stored information. The GET API improves performance by allowing fast data retrieval and supports secure access to user-specific information in the “Freshers Guide” project.

9. 2 POST API (Create Operation)

Purpose:

The POST API is used to send user data from the client (frontend) to the server (backend). It is mainly used when a user submits new information that needs to be stored in the database. This API plays an important role in creating new records and processing user authentication securely. It is commonly used for registration, login, and other form submissions where data needs to be sent to the server.

Used in Project:

- User Registration
- User Login
- Saving new user details in the database
- Submitting form data from frontend to backend

Example Endpoints:

- POST /api/register
- POST /api/login
- POST /api/feedback

Working:

When a new user registers in the "Freshers Guide" system, they fill out a form containing details such as name, email, password, and other required information. After submission, the frontend sends a POST request to the backend containing this data in JSON format. The backend receives the request, validates the input data, and checks for any errors such as missing fields or invalid format. If the data is valid, it is stored securely in the MySQL database as a new user record.

For the login process, the user enters their email and password. The frontend sends a POST request to the backend with these credentials. The backend verifies the details by comparing them with the data stored in the database. If the credentials match, the user is authenticated successfully, and a response is sent back to the frontend, often including

a success message or authentication token. If the credentials are incorrect, an error message is returned.

Overall, the POST API ensures secure data submission, user authentication, and proper communication between the frontend, backend, and database in the “Freshers Guide” project.

8.3 PUT API

Purpose:

The PUT API is used to update existing data in the database. It is mainly used when a user wants to modify or edit previously saved information. This API replaces the old data with new updated data and ensures that the database always contains the latest and correct information. It plays an important role in maintaining accurate and up-to-date user records.

Used in Project:

- Update resume details
- Edit user profile information
- Modify education, skills, or experience data
- Update contact details or personal information
- Edit previously saved interview preparation data

Example Endpoints:

- PUT /api/resumes/{id}
- PUT /api/users/{id}
- PUT /api/profile/{id}
- PUT /api/skills/{id}

Working:

When a user wants to edit existing information such as resume details or profile data in the "Freshers Guide" system, they make changes through the frontend interface. After editing, the frontend sends a PUT request to the backend containing the updated data along with a unique identifier (such as user ID or resume ID). The backend receives this request, validates the updated information, and then locates the existing record in the MySQL database. Once found, the old data is replaced with the new updated data. After successful update, the backend sends a response back to the frontend confirming that the changes have been saved successfully, along with the updated record.

8.3 DELETE API (Delete Operation)

Purpose:

The DELETE API is used to remove existing data from the database. It is mainly used when a user wants to permanently delete their information from the system. This API ensures that unwanted or unnecessary records are removed, helping to maintain a clean and efficient database.

Used in Project:

- Delete user account
- Remove user profile data
- Delete resume information
- Remove saved or outdated records (if required in future features)
- Maintain database cleanliness and data accuracy

Working:

When a user decides to delete their account in the "Freshers Guide" system, the frontend sends a DELETE request to the backend along with the user's unique ID. The backend receives the request and first validates whether the user exists in the system. After verification, it connects to the MySQL database and permanently removes the corresponding user record. Once the deletion process is completed successfully, the backend sends a confirmation response back to the frontend. The user is then logged out and their data is no longer available in the system.

Overall, the DELETE API ensures proper data management by allowing secure removal of unnecessary or unwanted records, helping maintain system efficiency and database integrity in the "Freshers Guide" project.

9. Advantages of the system

- Provides a simple and user-friendly interface for easy navigation.
- Offers structured and step-by-step career guidance for fresh graduates.
- Helps in resume building, interview preparation, and communication skill development.
- Includes comprehensive interview preparation with HR, technical, and common questions.
- Provides model answers, interview tips, checklists, and common mistakes.
- Features an interactive resume builder with preview and download option.
- Includes salary calculator, CTC breakdown, and negotiation guidance.
- Builds confidence by reducing confusion before entering the corporate world.
- Acts as an all-in-one career preparation platform.
- Saves time by providing all essential resources in a single system.
- Developed using Spring Boot and MySQL for scalability and real-world application.
- Helps students transition smoothly from academic life to professional careers.

10. Future Scope

- The Resume Builder module can be enhanced to provide advanced templates and download options (PDF/Word)
- The Interview Preparation module can be improved by adding more questions, mock tests, and real-time interview practice
- The Communication Skills module can include video-based learning and interactive exercises
- The Salary module can be expanded to provide real-time salary insights and company-wise salary comparison
- The Work Culture module can include real corporate case studies and practical examples
- The Office Etiquette module can be improved with interactive scenarios and quizzes
- A user login dashboard can be enhanced to store user progress and personalized data
- The system can be upgraded into a mobile application for better accessibility
- Additional features like notifications and reminders can be added for better user engagement

11. Conclusion

The **Freshers Guide** project is a well-structured career guidance platform designed to support students and fresh graduates in preparing for professional life. It effectively addresses the common challenges faced by freshers, such as lack of knowledge about resume building, interview preparation, communication skills, salary structure, and workplace behavior.

The system provides organized and easy-to-understand content through various modules like Resume Building, Interview Preparation, Communication Skills, Salary, Work Culture, and Office Etiquette. These modules help users gain both theoretical knowledge and practical guidance required for entering the corporate world.

The use of modern technologies such as React for frontend, Spring Boot with Java for backend, and MySQL for database ensures that the application is efficient, scalable, and user-friendly. The integration of full-stack development makes the project suitable for real-world applications as well as academic evaluation.

Overall, the Freshers Guide project not only helps users prepare for job opportunities but also builds confidence and awareness about professional life. It acts as a complete learning platform that bridges the gap between academic knowledge and industry expectations.

12. References

- W3Schools – https://www.w3schools.com
- GeeksforGeeks – https://www.geeksforgeeks.org
- MySQL Official Documentation – https://dev.mysql.com/doc/
- Stack Overflow – https://stackoverflow.com
- Spring Boot Official Documentation – https://spring.io/projects/spring-boot
- Oracle Java Documentation – https://docs.oracle.com/javase/